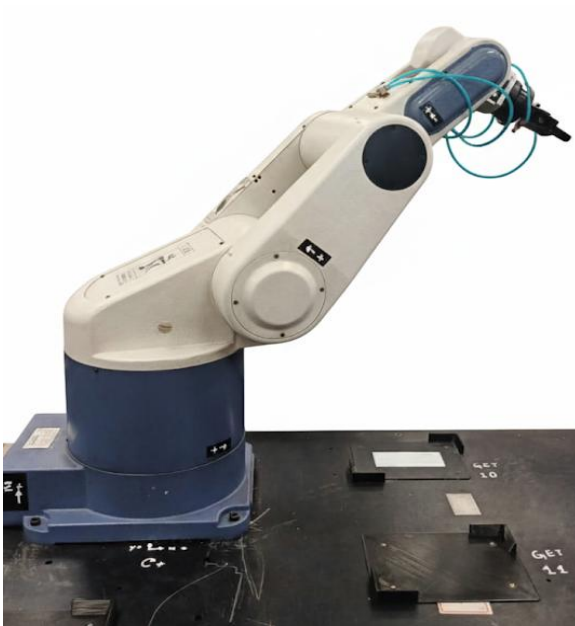


INFORME

PRÁCTICA PROFESIONAL SUPERVISADA PPS

DESARROLLO DE SOFTWARE Y PROGRAMACIÓN DE ROBOT DENTRO DEL MARCO DEL PROYECTO DE LABORATORIO DE MANUFACTURA FLEXIBLE



INTEGRANTES:

Carolina Vasquez

Lucas Montes

Pablo Garro Medina

PROFESORES:

Martin Gonzalez

Matías Hirak

Alejandro Simoncelli

Mauro Maceda

Cristian Lukaszewicz

Contenido

ÍNDICE

INFORMACIÓN	3
LABORATORIO CIM	3
ROBOT	4
CONTROLADOR	5
PROGRAMACIÓN	5
ACONDICIONAMIENTO DE ROBOT Y ÁREA DE TRABAJO	6
Acondicionamiento del área de trabajo	6
Adecuación de la mesa de trabajo	6
Distribución en mesa de trabajo	7
Diagnóstico y conexión del sistema de sujeción (gripper)	8
PIEZAS A ENSAMBLAR	9
PROGRAMACIÓN DE ROBOT	10
Programas GET y PUT	10
Programa principal LOBBY	11
Programa de ensamblaje PR1	12
Programa de ensamblaje PR2	15
Programa de ensamblaje PR3	18
Programa de ensamblaje PR4	20
Programa de ensamblaje PRES	23
CONCLUSIONES:	25

INFORMACIÓN

LABORATORIO CIM



Autor: imagen de elaboración propia



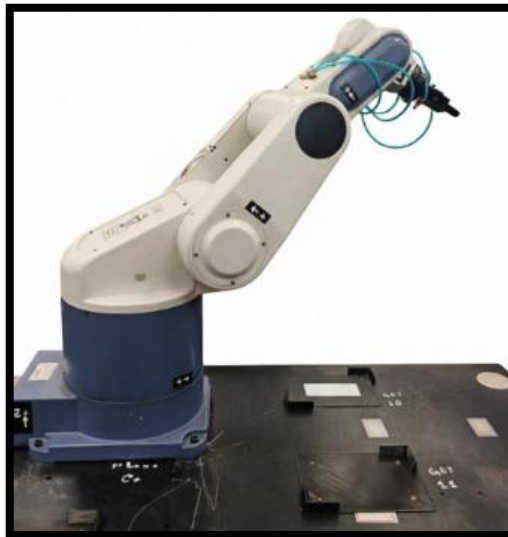
Autor: imagen de elaboración propia



Autor: imagen de elaboración propia

ROBOT

- Modelo: Performer mk3.
- Payload (carga máxima) 2 – 3 kg.
- Peso 32 kg.
- Grados de libertad: 5 rotativas.
- Gripper neumático.



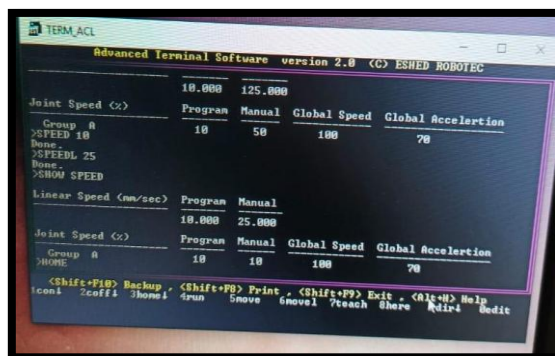
Autor: imagen de elaboración propia

CONTROLADOR PERFORMER AC YASKAWA



Autor: imagen de elaboración propia

PROGRAMACIÓN
Mediante una terminal en pc (ACL) o a través del
Teach Pendant.



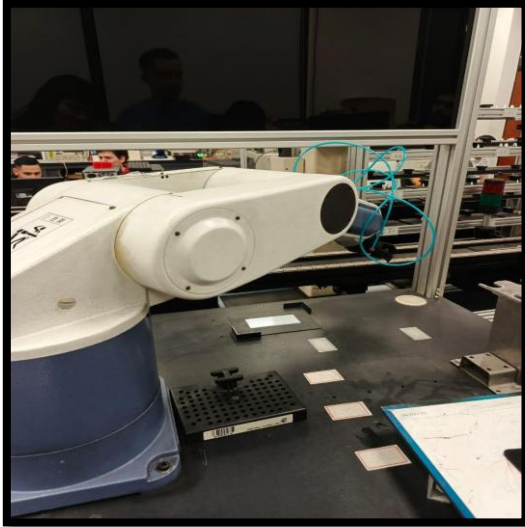
Autor: imagen de elaboración propia

ACONDICIONAMIENTO DE ROBOT Y ÁREA DE TRABAJO

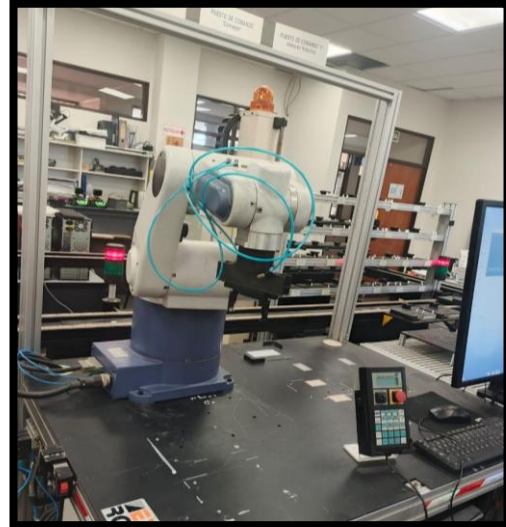
Acondicionamiento del área de trabajo

- Retiramos una pantalla de soldadura que obstruía el espacio.
- Reubicamos el robot con el fin de que pudiera alcanzar correctamente la estación de trabajo asignada.

ANTES



DESPUÉS

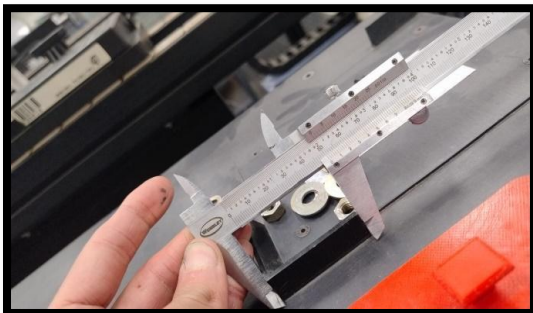


Autor: imagen de elaboración propia

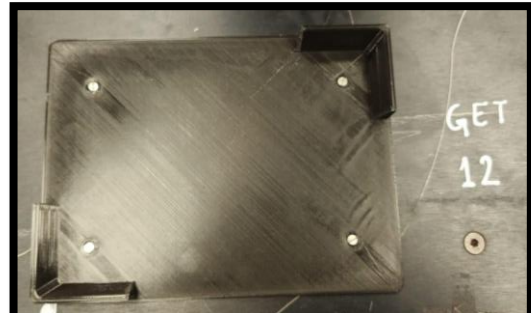
Adecuación de la mesa de trabajo

- Perforamos la superficie de la mesa para instalar nuevas bases de sujeción.
- Tomamos las dimensiones necesarias de dichas bases.
- Modelamos las bases en el software SolidWorks y posteriormente las fabricamos mediante impresión 3D.
- Una vez finalizadas, instalamos las bases en la mesa para asegurar el robot en la nueva posición.

RELEVAMIENTO

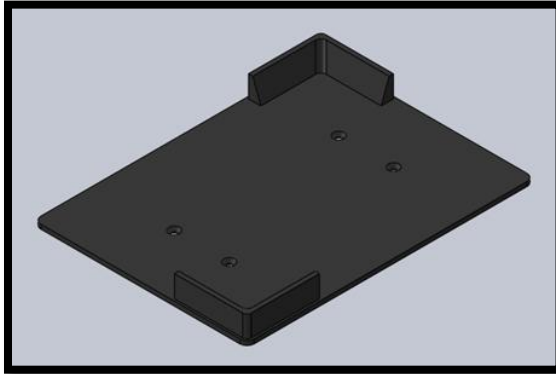


IMPRESIÓN



Autor: imagen de elaboración propia

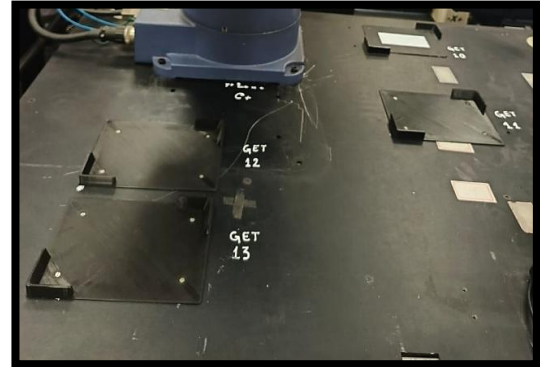
Modelado en 3D



Autor: imagen extraída de Software

CAD SolidWorks

Instalación de bases

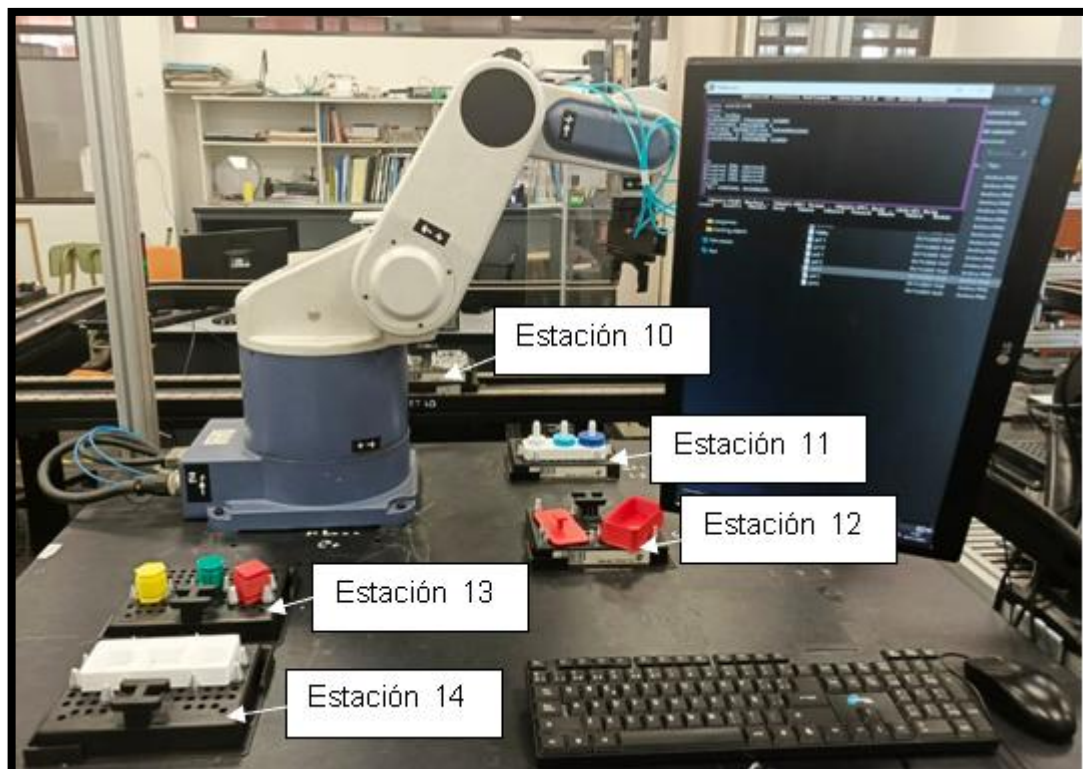


Autor: imagen de elaboración propia

Distribución en mesa de trabajo

El área de trabajo del robot cuenta con una estación en donde el carro correspondiente se estaciona que se la denomino como estación n°10.

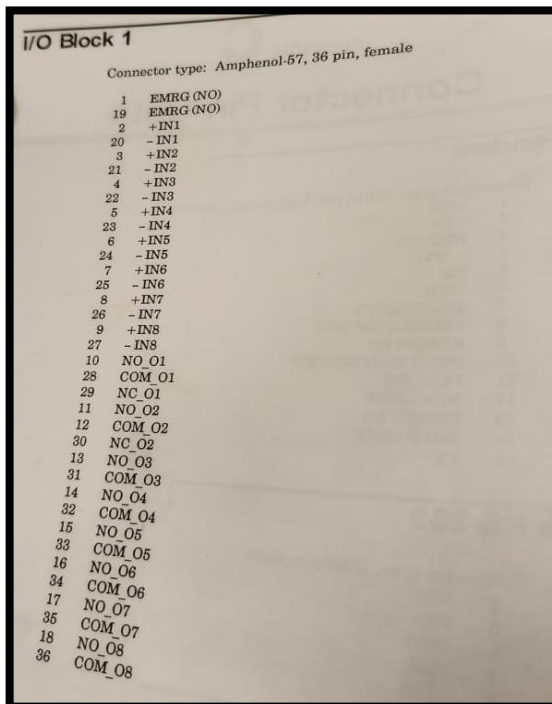
La mesa de trabajo cuenta con cuatro estaciones numeradas del 11 al 14 como muestra en la imagen.



Diagnóstico y conexión del sistema de sujeción (gripper)

- Detectamos que el robot no accionaba el gripper.
- Para resolverlo, identificamos las entradas y salidas correspondientes en el controlador del robot.
- Realizamos las conexiones necesarias entre el controlador y la electroválvula encargada de accionar el gripper.
- Con estas correcciones, el sistema de sujeción quedó operativo.

Identificación de entradas



Pruebas de entradas y salidas

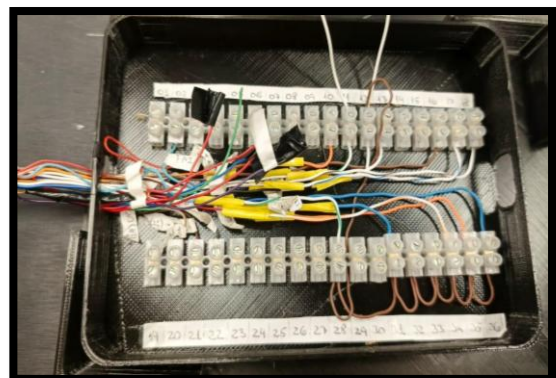


Electroválvula



Autor: imagen de elaboración propia

Modelado e impresión de caja de Cableado de entradas y salidas bornes



Autor: imagen de elaboración propia

PIEZAS A ENSAMBLAR

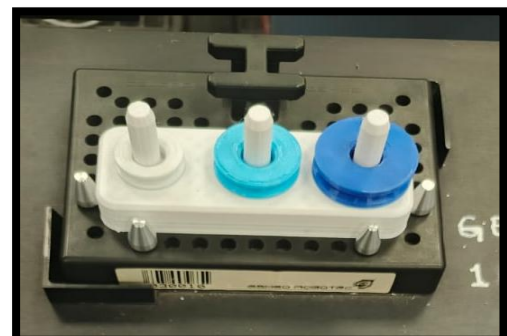
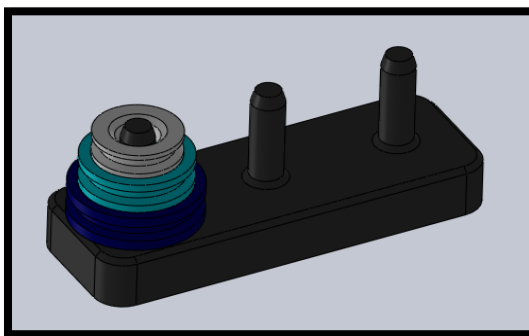
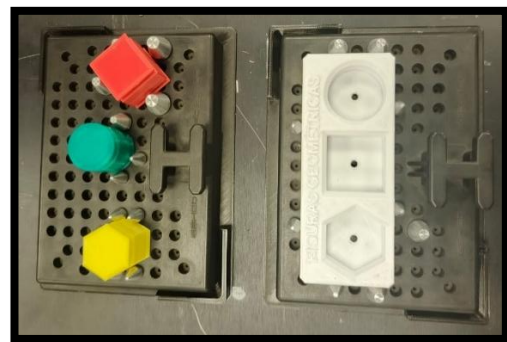
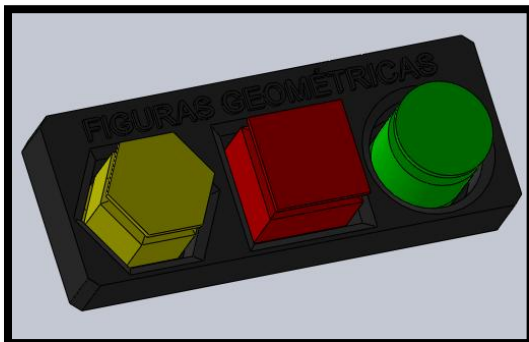
Para los ensamblajes de piezas se utilizaron piezas ya existentes del laboratorio y se diseñaron piezas adicionales.

Piezas existentes:



Autor: imagen de elaboración propia

Piezas diseñadas e impresas en 3D:



Autor: imagen de elaboración propia y de SolidWorks

PROGRAMACIÓN DE ROBOT

Programas GET y PUT

- Creamos en principio un vector principal llamado CIM2[50] el cual posee las ubicaciones para todos los GETXX y PUTXX que nos permiten tomar y dejar las bases en las 5 estaciones.

```

PROGRAM   GET10
*****
1339: SPEED      15
1340: SPEEDL     10
1341: MOVED      CIM2[1]
1342: OPEN
1343: MOVED      CIM2[2]
1344: MOVED      CIM2[3]
1345: MOVELD     CIM2[4]
1346: CLOSE
1347: MOVELD     CIM2[3]
1348: SPEEDL     25
1349: MOVELD     CIM2[2]
1350: END
  
```

```

PROGRAM   GET11
*****
1352: SPEED      15
1353: SPEEDL     10
1354: OPEN
1355: MOVED      CIM2[1]
1356: MOVED      CIM2[5]
1357: MOVED      CIM2[6]
1358: MOVELD     CIM2[7]
1359: CLOSE
1360: MOVELD     CIM2[6]
1361: SPEEDL     25
1362: MOVELD     CIM2[5]
1363: END
  
```

```

PROGRAM   GET12
*****
1365: SPEED      15
1366: SPEEDL     10
1367: OPEN
1368: MOVED      CIM2[1]
1369: MOVED      CIM2[8]
1370: MOVED      CIM2[9]
1371: MOVELD     CIM2[10]
1372: CLOSE
1373: MOVELD     CIM2[9]
1374: SPEEDL     25
1375: MOVELD     CIM2[8]
1376: END
  
```

```

PROGRAM   GET13
*****
1391: SPEED      15
1392: SPEEDL     10
1393: OPEN
1394: MOVED      CIM2[1]
1395: MOVED      CIM2[11]
1396: MOVED      CIM2[12]
1397: MOVELD     CIM2[13]
1398: CLOSE
1399: MOVELD     CIM2[12]
1400: SPEEDL     25
1401: MOVELD     CIM2[11]
1402: END
  
```

```

PROGRAM   GET14
*****
1417: SPEED      15
1418: SPEEDL     10
1419: OPEN
1420: MOVED      CIM2[1]
1421: MOVED      CIM2[14]
1422: MOVED      CIM2[15]
1423: MOVELD     CIM2[16]
1424: CLOSE
1425: MOVELD     CIM2[15]
1426: SPEEDL     25
1427: MOVED      CIM2[14]
1428: END
  
```

```

PROGRAM   PUT10
*****
1313: SPEED      15
1314: SPEEDL     10
1315: MOVED      CIM2[1]
1316: MOVED      CIM2[2]
1317: SPEEDL     25
1318: MOVELD     CIM2[3]
1319: SPEEDL     10
1320: MOVELD     CIM2[4]
1321: OPEN
1322: MOVELD     CIM2[3]
1323: MOVED      CIM2[2]
1324: END
  
```

```

PROGRAM   PUT11
*****
1326: SPEED      10
1327: SPEEDL     10
1328: MOVED      CIM2[1]
1329: MOVED      CIM2[5]
1330: SPEEDL     25
1331: MOVELD     CIM2[6]
1332: SPEEDL     10
1333: MOVELD     CIM2[7]
1334: OPEN
1335: MOVELD     CIM2[6]
1336: MOVED      CIM2[5]
1337: END
  
```

```

PROGRAM   PUT12
*****
1378: SPEED      15
1379: SPEEDL     10
1380: MOVED      CIM2[1]
1381: MOVED      CIM2[8]
1382: SPEEDL     25
1383: MOVELD     CIM2[9]
1384: SPEEDL     10
1385: MOVELD     CIM2[10]
1386: OPEN
1387: MOVELD     CIM2[9]
1388: MOVED      CIM2[8]
1389: END
  
```

PROGRAM PUT13		PROGRAM PUT14	
*****		*****	
1404:	SPEED 15	1430:	SPEED 15
1405:	SPEEDL 10	1431:	SPEEDL 10
1406:	MOVED CIM2[11]	1432:	MOVED CIM2[11]
1407:	MOVED CIM2[11]	1433:	MOVED CIM2[14]
1408:	SPEEDL 25	1434:	SPEEDL 25
1409:	MOVELD CIM2[12]	1435:	MOVELD CIM2[15]
1410:	SPEEDL 10	1436:	SPEEDL 10
1411:	MOVELD CIM2[13]	1437:	MOVELD CIM2[16]
1412:	OPEN	1438:	OPEN
1413:	MOVELD CIM2[12]	1439:	MOVELD CIM2[15]
1414:	MOVED CIM2[11]	1440:	MOVED CIM2[14]
1415:	END	1441:	END

Autor: imágenes de terminal del controlador ACL

Programa principal LOBBY

Creamos un programa principal llamado lobby que contiene los programas de ensamblaje PR1, PR2, PR3 y PR4

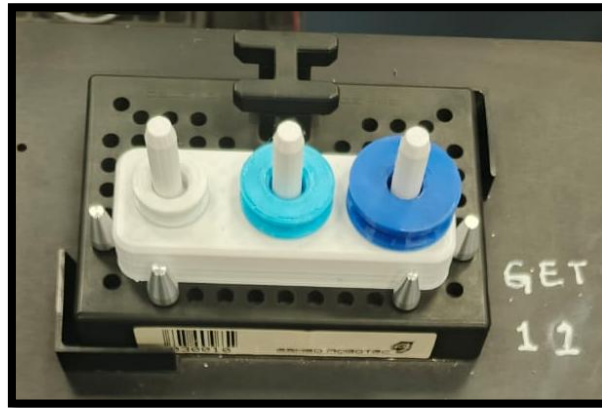
PROGRAM LOBBY	

1410:	PRINTLN "EJECUTANDO PROGRAMA LOBBY"
1411:	RUN PR1
1412:	RUN PR2
1413:	RUN PR3
1414:	RUN PR4
1415:	END

Autor: imagen de terminal del controlador ACL

Programa de ensamblaje PR1

El primer programa se trata del ensamblaje de piezas circulares diseñadas por nuestro equipo.



Autor: imagen de elaboración propia

Creamos el vector CIR2 que contiene las posiciones necesarias para realizar el ensamblaje en la estación 11 utilizando el programa ECIR2.

```

PROGRAM   ECIR2
*****
1505: SPEED      15
1506: SPEEDL     20
1507: MOVED      CIM2[1]
1508: OPEN
1509: MOVED      CIR2[1]
1510: MOVED      CIR2[2]
1511: MOVELD     CIR2[3]
1512: MOVELD     CIR2[4]
1513: CLOSE
1514: MOVELD     CIR2[3]
1515: MOVELD     CIR2[2]
1516: MOVELD     CIR2[5]
1517: MOVELD     CIR2[6]
1518: MOVELD     CIR2[7]
1519: OPEN
1520: MOVELD     CIR2[6]
1522: MOVED      CIR2[8]
1523: MOVELD     CIR2[9]
1524: MOVELD     CIR2[10]
1525: CLOSE
1526: MOVELD     CIR2[9]
1527: MOVELD     CIR2[8]
1528: MOVELD     CIR2[5]
1529: MOVELD     CIR2[6]
1530: MOVELD     CIR2[7]
1531: OPEN
1532: MOVELD     CIR2[6]
1533: MOVELD     CIR2[5]
1534: MOVED      CIR2[1]
1535: MOVED      CIM2[1]
1536: PRINTLN    "CIRCULOS ENSAMBLADOS"
1537: END
  
```

Autor: imagen de terminal del controlador ACL

Luego en el programa PR1 esperamos una señal del tablero de mando que en este caso es la entrada 1, que nos indicara que el carro correspondiente se detuvo en la estación 10 , Luego detenemos los otros programas que se están ejecutando.

Ya que el ensamblaje se realizará en la estación 11 se ejecutará el programa GET10 para que el robot tome la base con las piezas del carro y luego se ejecutara el programa PUT11 que colocara la base en la estación 11.

Luego se ejecutará el programa ECIR2 en donde el robot tomará el cilindro mediano de color celeste y lo apilará en el cilindro de diámetro mayor para luego tomar el cilindro mas pequeño de color blanco y apilarlo encima del cilindro celeste.

Una vez que ejecutado el programa ECIR2 se ejecutaran los programas GET11 Y PUT10 para colocar la base con las piezas ensambladas en el carro y finalmente enviamos una señal por la salida 1 para informarle al tablero de mando que el carro puede continuar circulando en la cinta.

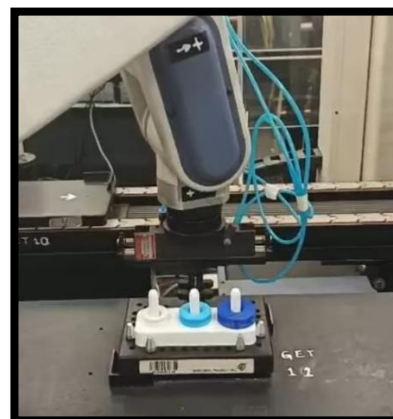
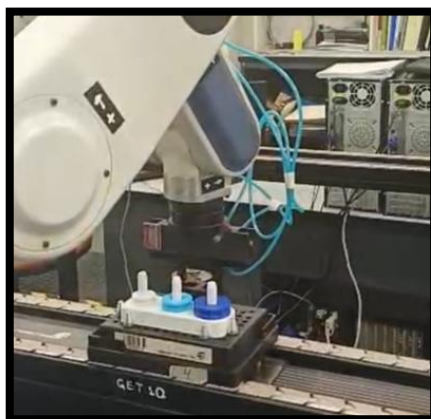
```

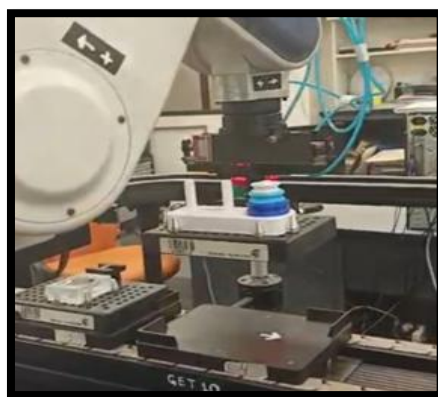
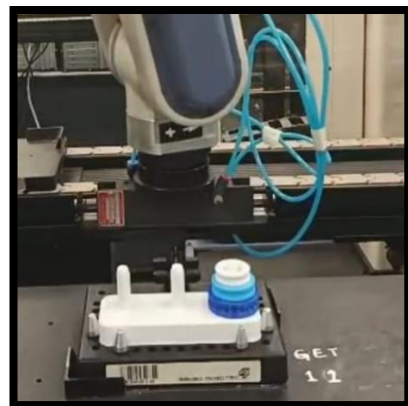
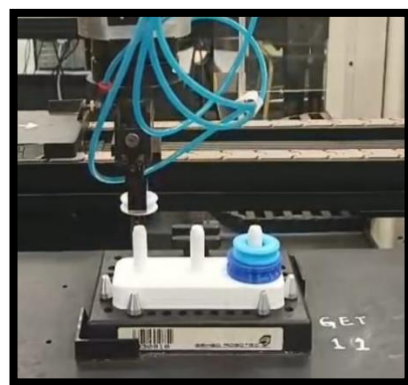
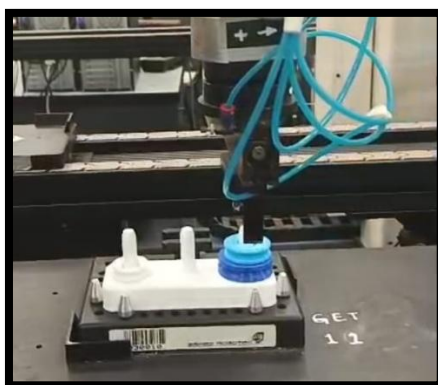
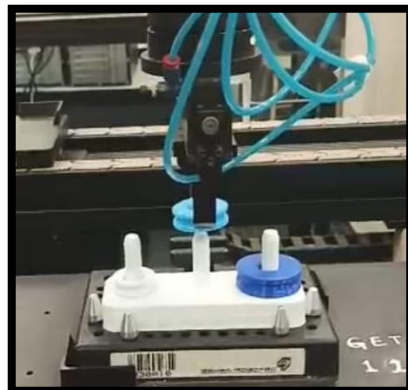
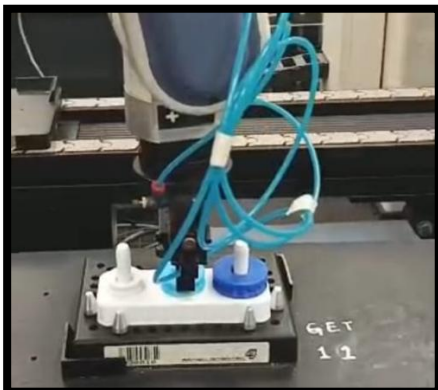
PROGRAM PR1
*****
1417: WAIT      IN[1] = 1
1419: WAIT      IN[1] = 0
1421: STOP      LOBBY
1422: STOP      PR2
1423: STOP      PR3
1424: STOP      PR4
1425: PRINTLN   "INICIANDO PROGRAMA 1"
1426: GOSUB     GET10
1427: GOSUB     PUT11
1428: GOSUB     ECIR2
1429: GOSUB     GET11
1430: GOSUB     PUT10
1431: SET       OUT[1] = 1
1433: PRINTLN   "PROGRAMA 1 TERMINADO"
1434: DELAY     50
1435: SET       OUT[1] = 0
1437: RUN       LOBBY
1438: END

```

Autor: imagen de terminal del controlador ACL

Secuencia del programa:





Autor: imagen de elaboración propia

Programa de ensamble PR2

El segundo programa se trata de el ensamble de una caja colocándole una tapa como se muestra a continuación.



Autor: imagen de elaboración propia

Creamos el vector VENS1 que contiene las posiciones necesarias para realizar el ensamble en la estación 12 utilizando el programa ENS12.

```

PROGRAM    ENS12
*****
1584: SPEED      15
1585: SPEEDL     20
1586: OPEN
1587: MOVED      CIM2[1]
1588: MOVED      VENS1[1]
1589: MOVED      VENS1[2]
1590: MOVELD     VENS1[3]
1591: CLOSE
1592: MOVELD     VENS1[2]
1593: MOVED      VENS1[4]
1594: MOVELD     VENS1[5]
1595: OPEN
1596: MOVELD     VENS1[4]
1597: MOVED      VENS1[1]
1598: MOVED      CIM2[1]
1599: PRINTLN    "CAJA ENSAMBLADA"
1600: END
  
```

Autor: imagen de terminal del controlador ACL

Luego en el programa PR2 esperamos una señal del tablero de mando que en este caso es la entrada 2, que nos indicara que el carro correspondiente se detuvo en la estación 10 , Luego detenemos los otros programas que se están ejecutando.

Ya que el ensamblaje se realizará en la estación 12 se ejecutará el programa GET10 para que el robot tome la base con las piezas del carro y luego se ejecutara el programa PUT12 que colocara la base en la estación 12.

Luego se ejecutará el programa ENS12 en donde el robot tomara la tapa de color rojo y la colocará en la caja.

Una vez que ejecutado el programa ENS12 se ejecutaran los programas GET12 Y PUT10 para colocar la base con las piezas ensambladas en el carro y finalmente enviamos una señal por la salida 1 para informarle al tablero de mando que el carro puede continuar circulando en la cinta.

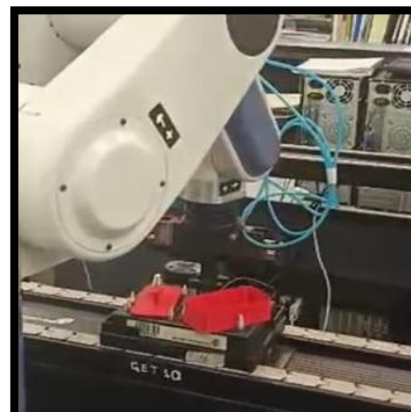
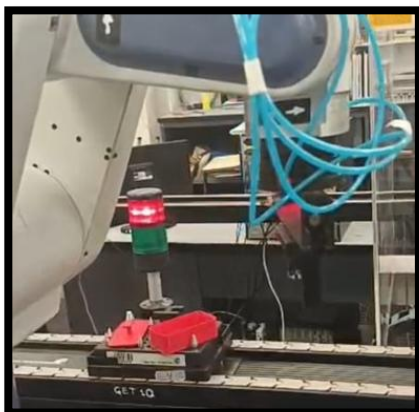
```

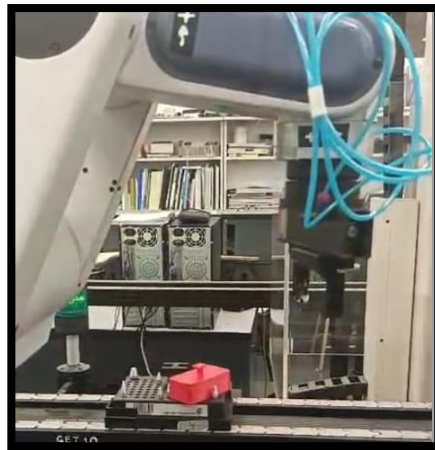
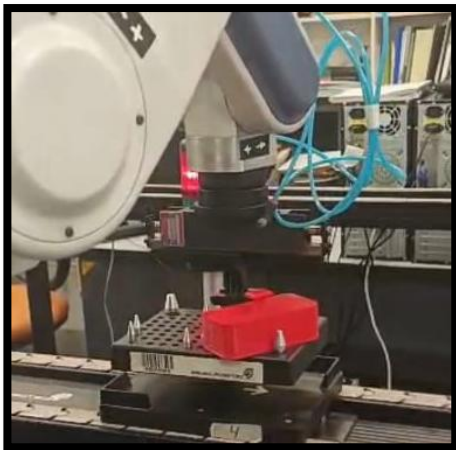
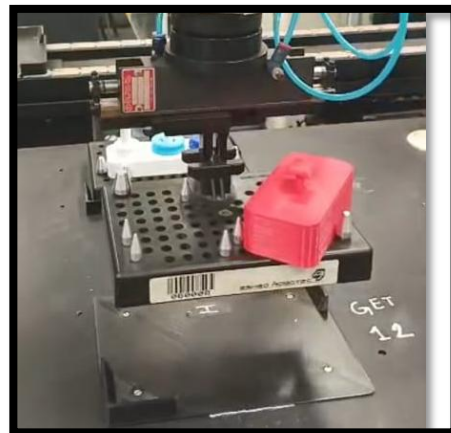
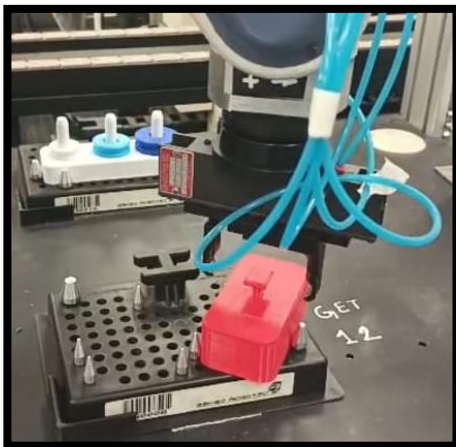
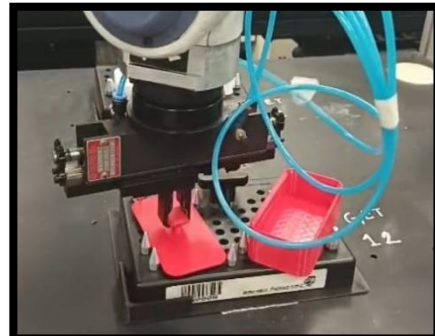
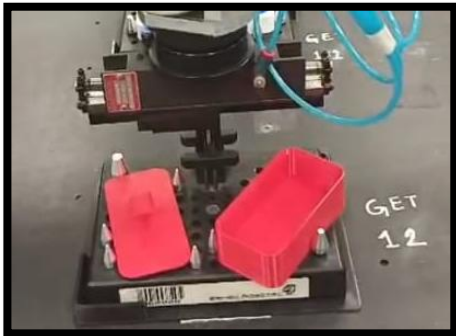
PROGRAM    PR2
*****
1440: WAIT      IN[2] = 1
1442: WAIT      IN[2] = 0
1444: STOP      LOBBY
1445: STOP      PR1
1446: STOP      PR3
1447: STOP      PR4
1448: PRINTLN    "INICIANDO PROGRAMA 2"
1449: GOSUB      GET10
1450: GOSUB      PUT12
1451: GOSUB      ENS12
1452: GOSUB      GET12
1453: GOSUB      PUT10
1454: SET        OUT[1] = 1
1456: PRINTLN    "PROGRAMA 2 TERMINADO"
1457: DELAY      50
1458: SET        OUT[1] = 0
1460: RUN        LOBBY
1461: END

```

Autor: imagen de terminal del controlador ACL

Secuencia del programa:





Autor: imagen de elaboración propia

Programa de ensamblaje PR3

El tercer programa se trata del ensamblaje del cilindro en la caja que se muestran a continuación.



Autor: imagen de elaboración propia

Creamos el vector VENS que contiene las posiciones necesarias para realizar el ensamblaje en la estación 10 utilizando el programa ENS10.

```

PROGRAM    ENS10
*****
1602: SPEED      15
1603: SPEEDL     15
1604: OPEN
1605: MOVED      CIM2[1]
1606: MOVED      VENS[1]
1607: MOVED      VENS[2]
1608: MOVELD     VENS[3]
1609: CLOSE
1610: MOVELD     VENS[2]
1611: MOVED      VENS[4]
1612: MOVELD     VENS[5]
1613: OPEN
1614: MOVELD     VENS[4]
1615: MOVED      VENS[1]
1616: MOVED      CIM2[1]
1617: PRINTLN    "ENSAMBLAJE DE CILINDRO CON CAJA TERMINAD"
1618: END

```

Autor: imagen de terminal del controlador ACL

Luego en el programa PR3 esperamos una señal del tablero de mando que en este caso es la entrada 3, que nos indicara que el carro correspondiente se detuvo en la estación 10, Luego detenemos los otros programas que se están ejecutando y ejecutamos el programa ENS10.

El robot tomara el cilindro y lo colocara en la caja, coordinando las velocidades y apertura y cierre del gripper .

Una vez que se ejecuto el programa ENS10 enviamos una señal por la salida 1 para informarle al tablero de mando que el carro puede continuar circulando en la cinta.

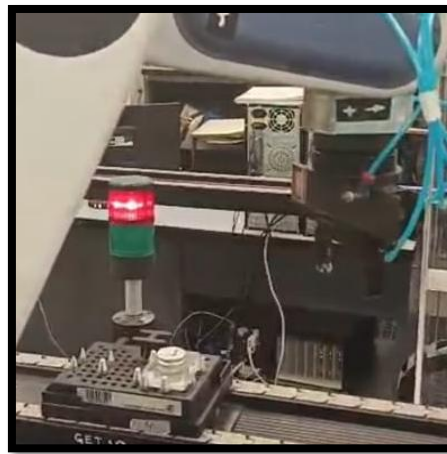
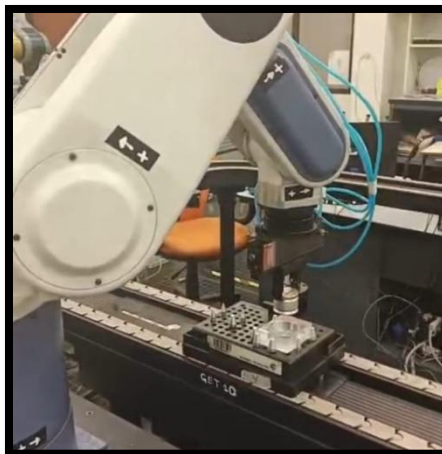
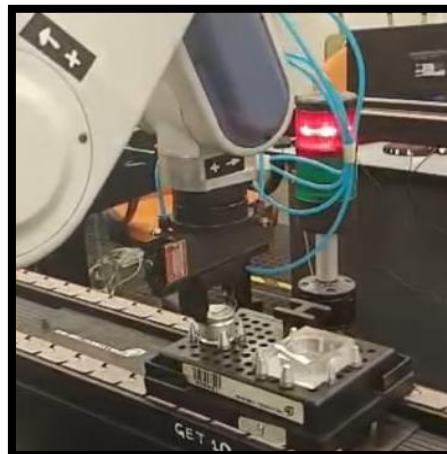
```

                                PROGRAM   PR3
                                *****
1463: WAIT                     INC[3] = 1
1465: WAIT                     INC[3] = 0
1467: STOP                     LOBBY
1468: STOP                     PR1
1469: STOP                     PR2
1470: STOP                     PR4
1471: PRINTLN                  "INICIANDO PROGRAMA 3"
1472: GOSUB                    ENS10
1473: SET                      OUT[1] = 1
1475: PRINTLN                  "PROGRAMA 3 TERMINADO"
1476: DELAY                    50
1477: SET                      OUT[1] = 0
1479: RUN                      LOBBY
1480: END

```

Autor: imagen de terminal del controlador ACL

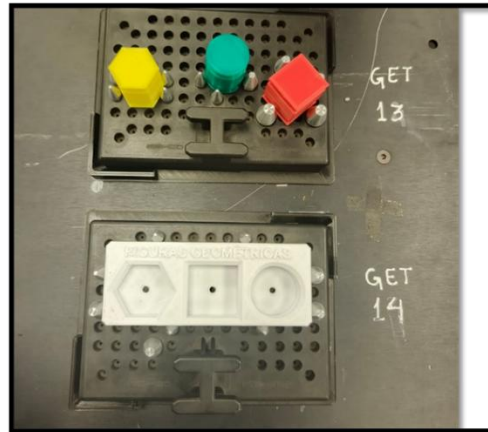
Secuencia del programa:



Autor: imagen de elaboración propia

Programa de ensamble PR4

El cuarto programa se trata del ensamble de figuras geométricas en una base con encastrados como se muestran a continuación.



Autor: imagen de elaboración propia

Creamos el vector VFIG que contiene las posiciones necesarias para realizar el ensamble en la estación 14 utilizando el programa EFIG.

```

PROGRAM EFIG
*****
1539: SPEED      15
1540: SPEEDL     20
1541: OPEN
1542: MOVED      CIM2[1]
1543: MOVED      UFIG[1]
1544: MOVED      UFIG[2]
1545: MOVELD     UFIG[3]
1546: CLOSE
1547: MOVELD     UFIG[2]
1548: MOVED      UFIG[8]
1549: MOVELD     UFIG[9]
1550: OPEN
1551: MOVELD     UFIG[8]
1552: MOVED      UFIG[6]
1553: MOVELD     UFIG[7]
1554: CLOSE
1556: MOVED      UFIG[4]
1557: MOVELD     UFIG[5]
1558: OPEN
1559: MOVELD     UFIG[4]
1560: MOVED      UFIG[10]
1561: MOVELD     UFIG[11]
1562: CLOSE
1563: MOVELD     UFIG[10]
1564: MOVED      UFIG[12]
1565: MOVELD     UFIG[13]
1566: OPEN
1567: MOVELD     UFIG[12]
1568: MOVED      UFIG[11]
1569: MOVED      CIM2[1]
1570: PRINTLN    "FIGURAS GEOMETRICAS ENSAMBLADAS"
1571: END
  
```

Autor: imagen de terminal del controlador ACL

Luego en el programa PR4 esperamos una señal del tablero de mando que en este caso es la entrada 4, que nos indicara que el carro correspondiente se detuvo en la estación 10, Luego detenemos los otros programas que se están ejecutando.

Ya que el ensamblaje se realizará en la estación 14 se ejecutará el programa GET10 para que el robot tome la base con las piezas del carro y luego se ejecutara el programa PUT14 que colocara la base en la estación 14.

Luego se ejecutará el programa EFIG en donde el robot primero tomara el cuadrado rojo y lo colocara en el encastre correspondiente, luego hará lo mismo con el cilindro y por último encastrara el hexágono.

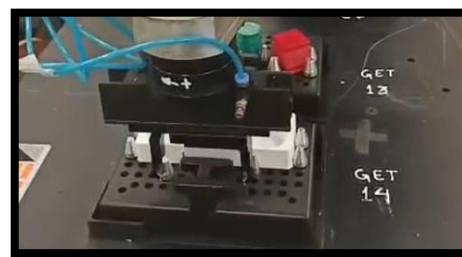
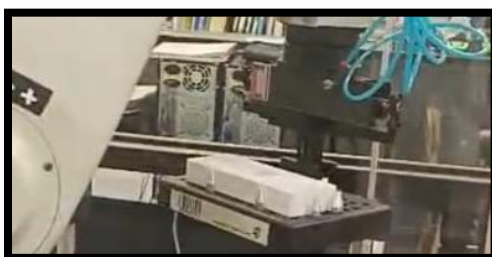
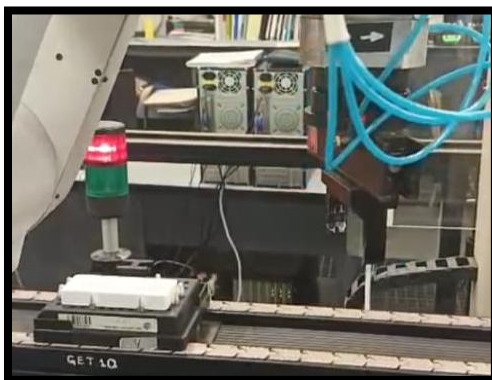
Una vez que iniciamos el programa EFIG se ejecutaran las sentencias GET14 y PUT10 para colocar la base con las piezas ensambladas en el carro y finalmente enviamos una señal por la salida 1 para informarle al tablero de mando que el carro puede continuar circulando en la cinta.

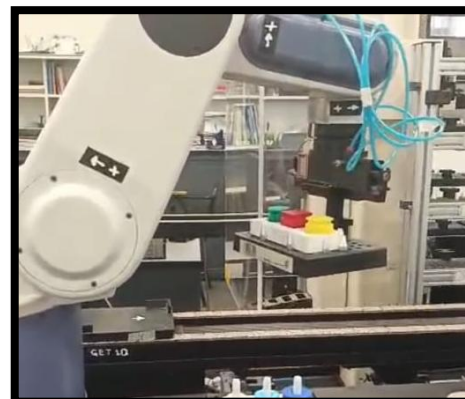
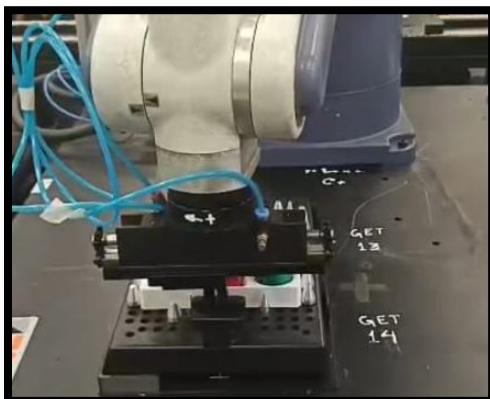
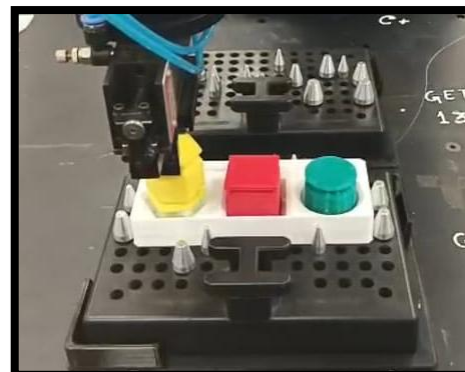
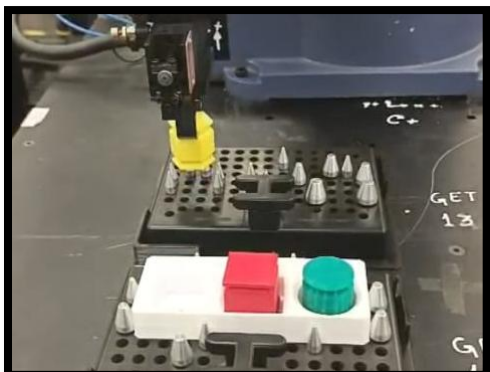
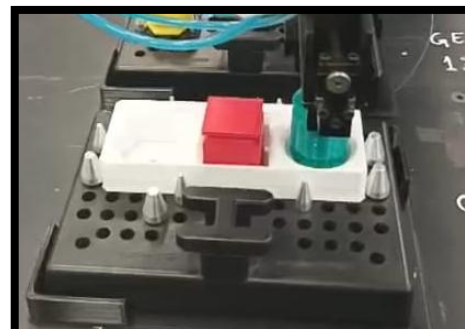
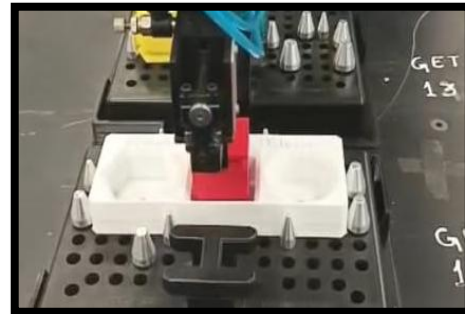
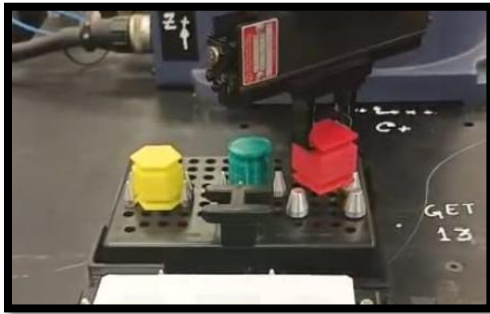
```

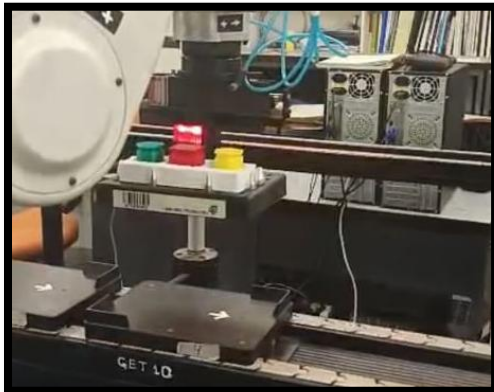
PROGRAM PR4
*****
1482: WAIT      IN[4] = 1
1484: WAIT      IN[4] = 0
1486: STOP      LOBBY
1487: STOP      PR1
1488: STOP      PR2
1489: STOP      PR3
1490: PRINTLN   "INICIANDO PROGRAMA 4"
1491: GOSUB     GET10
1492: GOSUB     PUT14
1493: GOSUB     EFIG
1494: GOSUB     GET14
1495: GOSUB     PUT10
1496: SET       OUT[1] = 1
1498: PRINTLN   "PROGRAMA 4 TERMINADO"
1499: DELAY     50
1500: SET       OUT[1] = 0
1502: RUN
1503: END

```

Secuencia del programa:







Autor: imágenes de elaboración propia

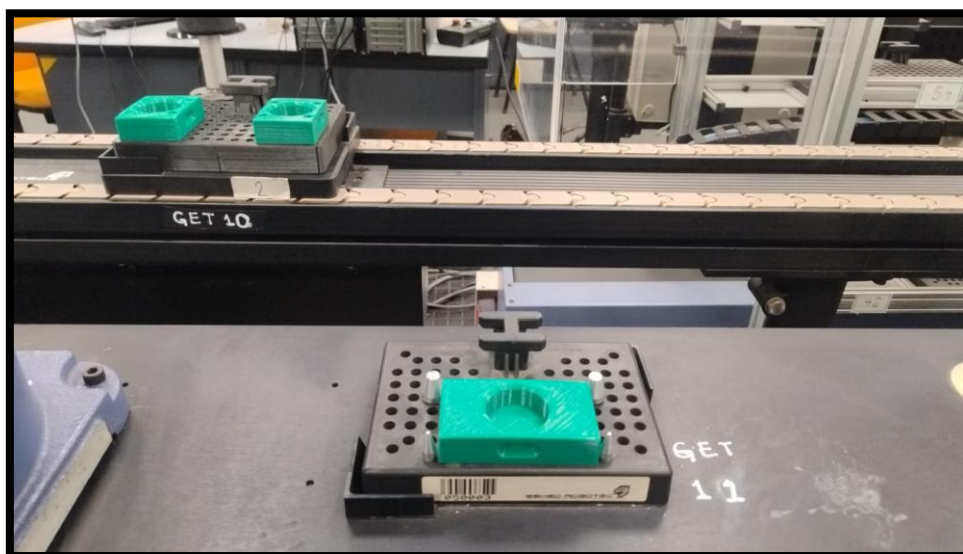
Programa de ensamble PRES

Cuando llega el carro a la estación 10 y el controlador envía la señal por la entrada 3 y se ejecuta automáticamente el programa PRES.

Este programa esta dentro de lobby por lo que no se podrán correr PRES y PR3 simultáneamente porque esperarían la misma entrada.

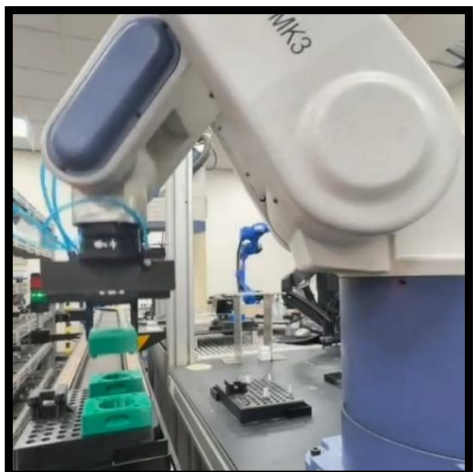
Se utiliza como una opción mas por lo que se elegirá que programa correr en el programa lobby (dejando solo el que se desea ejecutar).

El carro llega a la estación 10 con dos piezas y el robot coloca la tercera pieza que se encuentra en la estación 11 por encima de las piezas existentes, luego enviamos una señal por la salida 1 para informarle al tablero de mando que el carro puede continuar circulando en la cinta.



Autor: imagen de elaboración propia

Secuencia del programa:



Autor: imágenes de elaboración propia

CONCLUSIONES:

El desarrollo de la Práctica Profesional Supervisada permitió integrar y aplicar en un entorno real de laboratorio los conocimientos adquiridos a lo largo de la carrera de Ingeniería Mecatrónica. Además de lograr la programación funcional del manipulador industrial asignado dentro del laboratorio CIM, el equipo llevó adelante un proceso integral que incluyó el reacondicionamiento de la estación de trabajo, el diseño de componentes en software CAD, el armado de la bornera y la confección de cableados para las entradas y salidas del sistema, garantizando una instalación segura, ordenada y estandarizada.

El proyecto se desarrolló de manera conjunta con los otros equipos del laboratorio, lo que implicó coordinar criterios, compartir recursos y unificar estándares. Esta dinámica permitió fortalecer habilidades blandas como la comunicación, la organización y la toma de decisiones en equipo, fundamentales para cualquier entorno industrial. Gracias a este trabajo colaborativo, se logró que todas las estaciones funcionaran de manera integrada bajo un tablero principal capaz de coordinar las secuencias de cada manipulador.

Asimismo, se ejecutaron pruebas de funcionamiento, simulaciones y secuencias de ensamblaje en conjunto con los manipuladores programados por los otros grupos. Este enfoque sistémico no solo validó la operación de cada estación, sino que también permitió evaluar la interacción entre ellas, acercando la práctica a un escenario industrial real. Finalmente, se registró toda la documentación técnica del proceso, de modo que futuros grupos puedan continuar, mejorar o escalar el trabajo realizado.

En síntesis, la PPS permitió poner en práctica herramientas de robótica, diseño mecánico, automatización, programación industrial y gestión de proyectos, consolidando tanto las capacidades técnicas como las competencias profesionales necesarias para el ejercicio de la ingeniería en entornos colaborativos y multidisciplinarios.